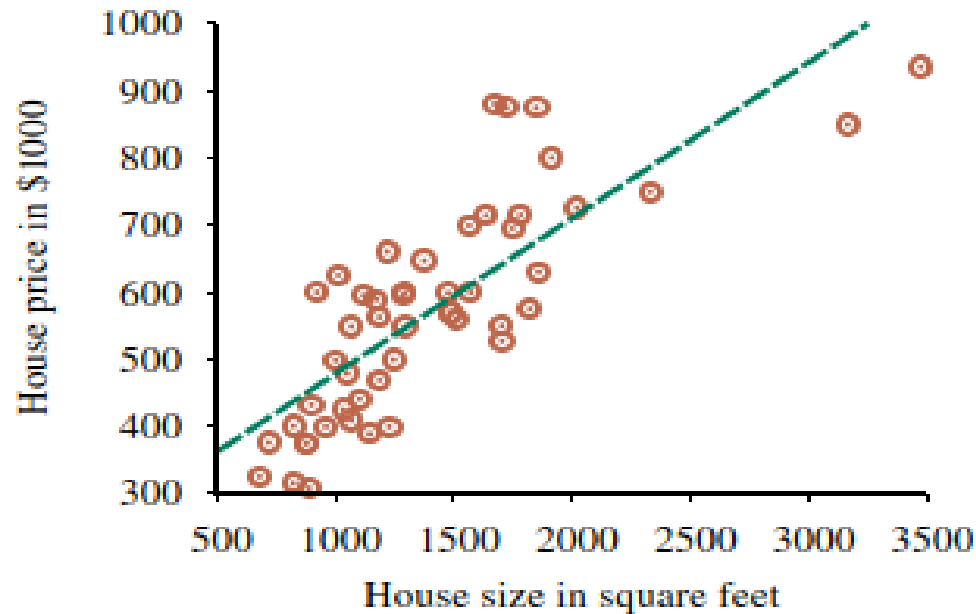


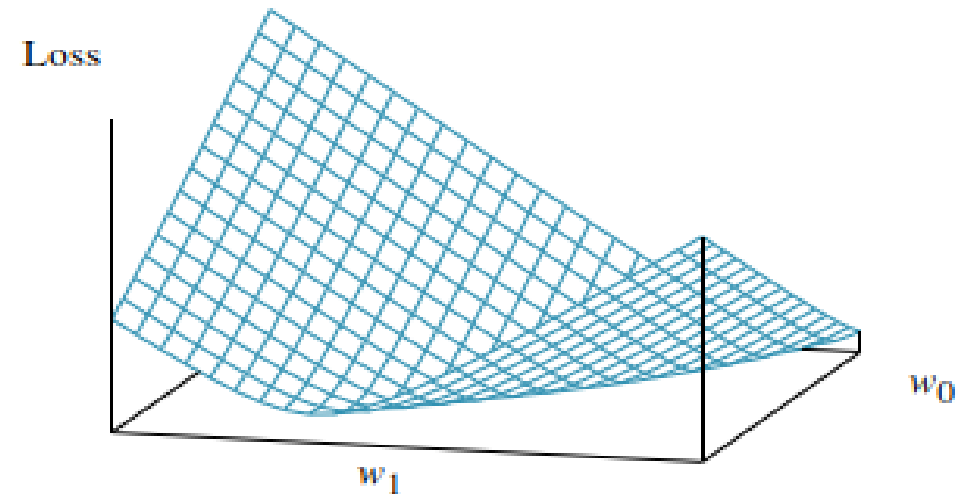
Learning From Examples

ML Basics

Linear Regression



(a)



(b)

Figure 19.13 (a) Data points of price versus floor space of houses for sale in Berkeley, CA, in July 2009, along with the linear function hypothesis that minimizes squared-error loss: $y = 0.232x + 246$. (b) Plot of the loss function $\sum_j (y_j - w_1 x_j + w_0)^2$ for various values of w_0, w_1 . Note that the loss function is convex, with a single global minimum.

Gradient descent

- Previous way was closed form.

$\mathbf{w} \leftarrow$ any point in the parameter space

while not converged do

for each w_i in $\mathbf{w}$ do

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$$

$$\begin{aligned} \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w}) &= \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x))^2 = 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - (w_1 x + w_0)). \end{aligned} \quad (19.5)$$

Applying this to both w_0 and w_1 we get:

$$\frac{\partial}{\partial w_0} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)); \quad \frac{\partial}{\partial w_1} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x.$$

Plugging this into Equation (19.4), and folding the 2 into the unspecified learning rate α , we get the following learning rule for the weights:

$$w_0 \leftarrow w_0 + \alpha (y - h_{\mathbf{w}}(x)); \quad w_1 \leftarrow w_1 + \alpha (y - h_{\mathbf{w}}(x)) \times x.$$

These updates make intuitive sense: if $h_{\mathbf{w}}(x) > y$ (i.e., the output is too large), reduce w_0 a bit, and reduce w_1 if x was a positive input but increase w_1 if x was a negative input.

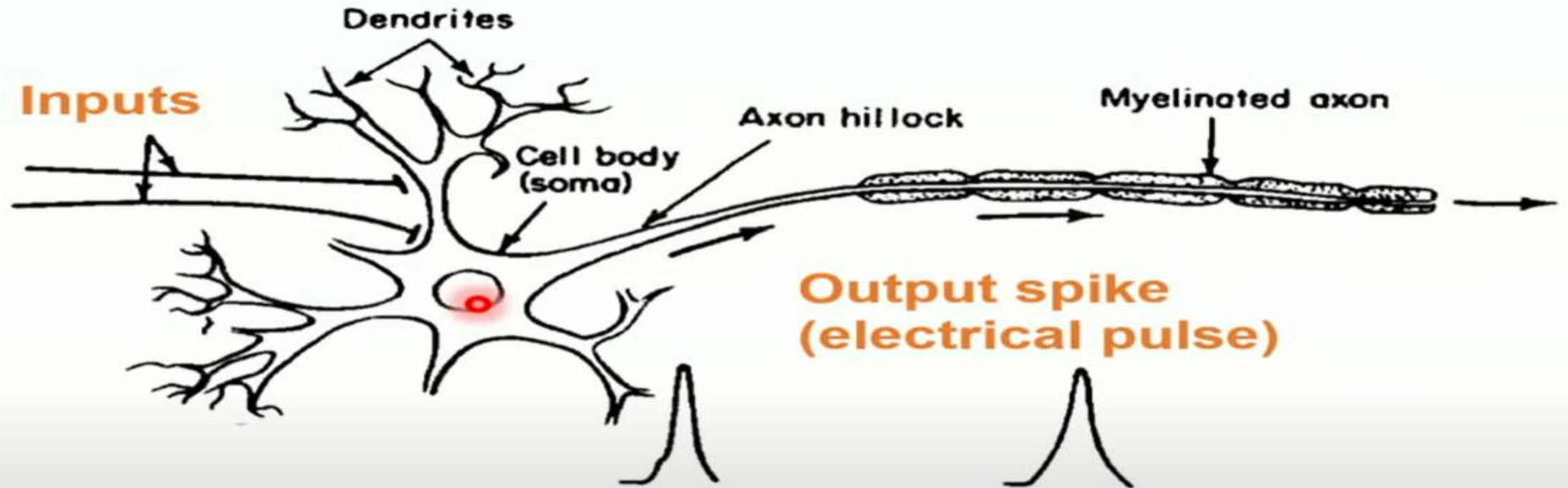
Linear Regression

The preceding equations cover one training example. For N training examples, we want to minimize the sum of the individual losses for each example. The derivative of a sum is the sum of the derivatives, so we have:

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)); \quad w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j.$$

Neural Networks

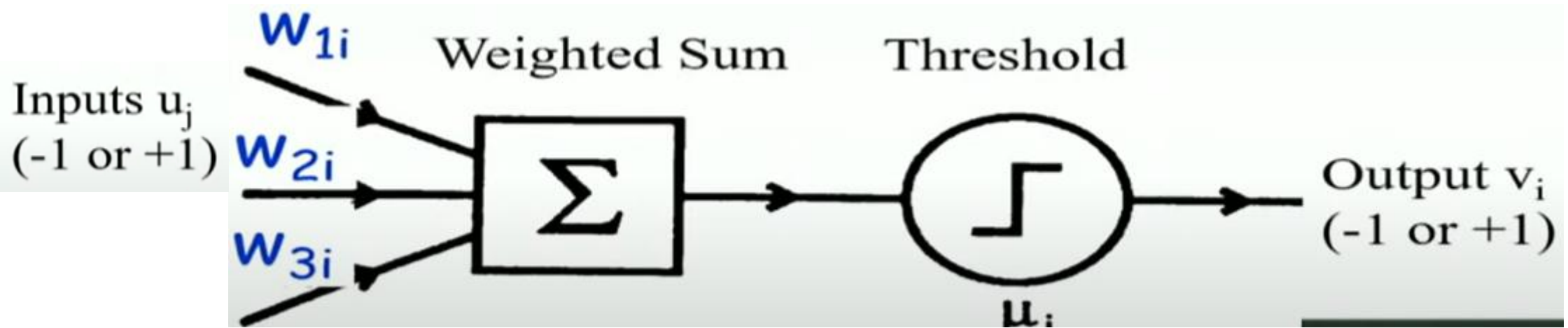
Neurons communicate via spikes



Output spike roughly dependent on whether sum of all inputs reaches a threshold

Neuron

- Artificial neuron:
 - m binary inputs (-1 or 1), 1 output (-1 or 1)
 - Synaptic weights w_{ji}
 - Threshold μ_i



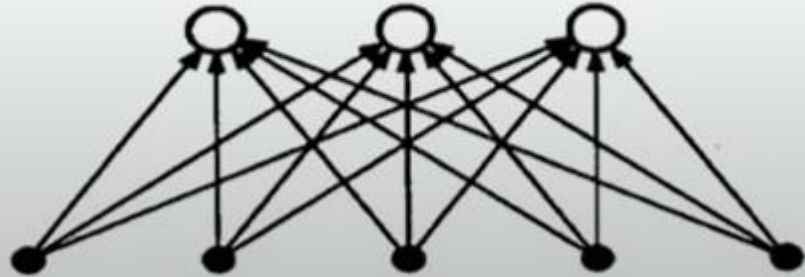
$$v_i = \Theta\left(\sum_j w_{ji} u_j - \mu_i\right)$$

$$\Theta(x) = 1 \text{ if } x > 0 \text{ and } -1 \text{ if } x \leq 0$$

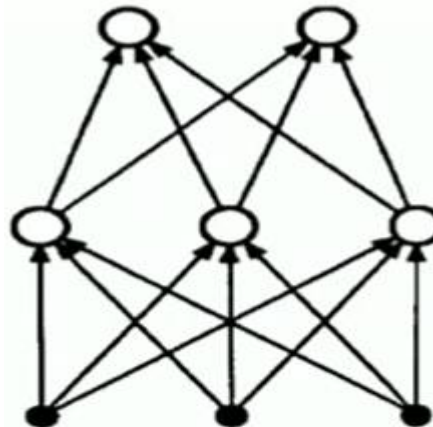
“Perceptrons” for Classification

- Fancy name for a type of layered “feed-forward” networks (no loops)
- Uses artificial neurons (“units”) with binary inputs and outputs

Single-layer



Multilayer



For Classification Problem:
Two classes: find linear separator that separates two class

Perceptrons and Classification

- Consider a single-layer perceptron
 - Weighted sum forms a *linear hyperplane*

$$\sum w_{ji} u_j - \mu_i = 0$$

- Everything *on one side* of this hyperplane is in **class 1** (output = +1) and everything *on other side* is **class 2** (output = -1)
- Any function that is linearly separable can be computed by a perceptron